ARE 212 Section 5:
Hypothesis Testing

Fiona Burlig

February 18, 2016

Problem set recap

In general, you guys did a great job on the problem set. One down! I've graded all of them. I've left everyone a grade, and for those of you who turned something in, some minor comments, and if necessary, some major comments, so do take a look at those on bCourses.[1] Max should post the answer key shortly.[2]

A few minor comments that apply to many of you (in no particular order):

- Lots of you had `knitr` chunk code that ran off the edge of the page, making it hard to know what commands you actually ran. Play around with carriage-returns in your code to fix this. `R` is smart enough to know that things like functions continue on the next line if you put linebreaks in sensible places (take a look at any of my sets of section notes for examples).
- There were a couple of instances where people were calculating residuals, DoF, and other quantities by hand. Wherever possible, it's worth writing this kind of thing into a function, since you'll do it over and over again.
- Friends don't let friends hard-code data or variables into functions. Any function you write should be perfectly general, and perform actions only on its inputs. What do I mean by that? If your function is something like:

```
myFXN <- function(data, y, X) {

}
```

  The stuff inside the curly braces should only use `data`, `y`, and `X` as inputs. Importantly, you **do not** want to have something like this:

```
myFXN <- function(data, y, X) {
  mean(thatotherdata) + select_(thatotherdata, "myvariable")
}
```

  where `thatotherdata` and `''myvariable''` come from outside the function environment.

[1] I know nobody likes to come into office hours *after* the problem set is done, but if you're struggling to understand the comments I gave you, or think that something went horribly wrong, please drop by!

[2] It's likely that Max isn't `dplyr` savvy, so the problem set may do things in a different way than we've been doing in section. As I keep saying, there are a bunch of ways to do things in `R`, and it's good to get exposure to all of them. The section notes contain *almost* all of the functions you needed to do the problem set in `dplyr` form.

- The easiest way to read CSV files into `R` is to use the `readr` package's `read_csv()` function, which automatically spits out a `tbl_df`.
- A couple of you aren't using `knitr`, but *are* using LaTeX. I highly recommend taking the time to learn `knitr` - it'll save you headaches later.
- FWL seemed to give some people some trouble. Take a look at last week's section notes for a refresher. This is an important theorem.
- Additively including a squared term in a linear regression *does not* violate our assumptions. A couple people had issues with the interpretation of this question.
- **The big kahuna:** A non-zero fraction of you got a singularity error when trying to run the regression of CO2 per capita on GDP per capita and GDP per capita squared. I think I've finally figured out why. It's not that GDPpc and GDPpc^2 are collinear - they're clearly not. It's that GDP per capita is a big number, and GDP per capita squared is an even bigger number. When you try to calculate $\hat{\beta}$ from this regression, you have to calculate $\mathbf{X}'\mathbf{X}^{-1}X'y$, where included in $\mathbf{X}$ is both GDPpc and GDPpc2. This means multiplying their values into an EVEN BIGGER number, that sets `R` on fire. I think this was happening to people who missed Max's suggestion to "keep both variables in the new units" at the end of Question 14.

Loose ends

As far as I know, there weren't questions left hanging from last week. But a couple of you have asked me for some data-cleaning tips on your problem set, so I'll provide a few small suggestions here.

One of the most important quantities to know how to handle is missing values. All data are imperfect.[3] One common imperfection is missing observations. Let's actually use the data from that problem set to explore a couple of things with missing data.

```
### SETUP
library(dplyr)
library(lfe)
library(readr)


### FXNS
tbldfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

OLS <- function(data, y, X) {
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
```

[3] A prominent environmental economist likes to say that data are like your worst boyfriend or girlfriend - they'll never fail to let you down.

```
  return(betahat)
}
```

Now load the data:

```
psData <- read_csv("psonedata.csv")

psData

## Source: local data frame [219 x 5]
##
##                country country_code        CO2         GDP
##                  (chr)        (chr)      (chr)       (chr)
## 1          Afghanistan          AFG    8470.77 10243250247
## 2              Albania          ALB   4415.068 10498760062
## 3              Algeria          DZA 119276.509 1.16511E+11
## 4       American Samoa          ASM         ..          ..
## 5              Andorra          ADO    517.047  2829050839
## 6               Angola          AGO  29743.037          ..
## 7  Antigua and Barbuda          ATG    524.381  1012089673
## 8            Argentina          ARG 179000.938 2.91705E+11
## 9              Armenia          ARM    4217.05  5918256387
## 10               Aruba          ABW    2456.89          ..
## ..                 ...          ...        ...         ...
## Variables not shown: Pop (int)
```

Uh oh - we had some importing problems here. We should have two string variables (`country` and `country_code`, and three numerical variables (`CO2, GDP`, and `Pop`). But instead, we have four string variables and only one numerical variable. This is another reason to like the `tbl_df` format - it displays the variable types at the top of the dataset. What happened? It looks like the CSV we read in had a missing value character (`..`) that we failed to take into account, so `R` had no choice but to read this dataset in as strings. Let's try that again:

```
psData <- read_csv("psonedata.csv", na = (".."))
psData

## Source: local data frame [219 x 5]
##
##                country country_code        CO2          GDP
##                  (chr)        (chr)      (dbl)        (dbl)
## 1          Afghanistan          AFG   8470.770  10243250247
## 2              Albania          ALB   4415.068  10498760062
## 3              Algeria          DZA 119276.509 116511000000
## 4       American Samoa          ASM         NA           NA
## 5              Andorra          ADO    517.047   2829050839
## 6               Angola          AGO  29743.037           NA
## 7  Antigua and Barbuda          ATG    524.381   1012089673
## 8            Argentina          ARG 179000.938 291705000000
## 9              Armenia          ARM   4217.050   5918256387
```

```
## 10               Aruba           ABW   2456.890           NA
## ..                 ...           ...        ...          ...
## Variables not shown: Pop (int)

# add an intercept
psData <- mutate(psData, ones = 1)
```

Much better. Notice that things are now numeric when they're supposed to be; the scientific notation has been read in correctly, and we've got a couple of `NA` values in our dataset, which are objects that `R` understands. Having these values in our dataset can get in the way of things, though. Let's try to, for example, calculate the mean GDP in our data (we can do this both the `dplyr` way and the regular (boring) way:

```
#dplyr way
(meanGDP <- summarize(psData, mean(GDP)))

## Source: local data frame [1 x 1]
##
##   mean(GDP)
##       (dbl)
## 1        NA

#boring way
(meanGDP <- mean(psData$GDP))

## [1] NA
```

Both of these return `NA`s - what gives?[4] Clearly the mean of GDP isn't actually `NA`. But `R` doesn't know how to add a missing value to a number, so the calculation is undefined. We'll have to deal with these missing values in some way. Many canned functions (including `mean()`) have built-in options for dealing with missing data, like this:

```
(meanGDP <- summarize(psData, mean(GDP, na.rm = TRUE)))

## Source: local data frame [1 x 1]
##
##   mean(GDP, na.rm = TRUE)
##                     (dbl)
## 1            275507512008

(meanGDP <- mean(psData$GDP, na.rm = TRUE))

## [1] 275507512008
```

The `na.rm` argument tells `mean()` to remove the `NA`s before calculating the mean. Clever, huh? This is great. But some functions are a little sneakier. Let's try to run a regression using `felm()` on our missing-value-containing dataset:

[4] Notice also that summarize is just using the built-in `mean()` function.

```
(felmOut <- felm(data = psData, GDP ~ CO2))

## (Intercept)         CO2
##   9.398e+10   1.108e+06
```

This works without us having to specifically tell it to deal with the `NA`s, which, on the one hand is nice, but on the other hand, is something to watch out for. Unlike Stata, whose base regression output will tell you how many observations were dropped, `felm()`'s standard output doesn't do this for us. We have to go to the summary to see this:

```
(fullFelmOut <- summary(felmOut))

##
## Call:
##    felm(formula = GDP ~ CO2, data = psData)
##
## Residuals:
##        Min         1Q     Median         3Q        Max
## -5.375e+12 -9.394e+10 -9.111e+10 -7.417e+10  7.513e+12
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 9.398e+10  5.942e+10   1.582    0.115
## CO2         1.108e+06  7.734e+04  14.325   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.909e+11 on 184 degrees of freedom
##   (33 observations deleted due to missingness)
## Multiple R-squared(full model): 0.5273   Adjusted R-squared: 0.5247
## Multiple R-squared(proj model): 0.5273   Adjusted R-squared: 0.5247
## F-statistic(full model):205.2 on 1 and 184 DF, p-value: < 2.2e-16
## F-statistic(proj model): 205.2 on 1 and 184 DF, p-value: < 2.2e-16
```

For the interested reader, note that we can actually get `felm()` to tell us *which* observations were dropped - this is stored in `na.action` - so we can take our mistrust of the function down a notch:

```
fullFelmOut$na.action

##   4   6  10  36  39  50  64  68  75  77  92 101 103 112 129 134
##   4   6  10  36  39  50  64  68  75  77  92 101 103 112 129 134
## 138 143 155 161 169 173 175 180 187 198 199 210 215 216 217 218
## 138 143 155 161 169 173 175 180 187 198 199 210 215 216 217 218
## 219
## 219
## attr(,"class")
## [1] "omit"
```

If we were to run this with our own OLS function:

```
myReg <- OLS(psData, "GDP", c("ones", "CO2"))
myReg

##      GDP
## ones  NA
## CO2   NA
```

Oops. That doesn't work. Since we'll be doing this (and every other) problem set with our own functions, let's handle the NAs. We can do this in a couple of ways:

```
noNAPSData <- na.omit(psData)
```

Notice what `R` did there - it omitted both observations with NAs in them. This makes sense from a regression perspective, in that we can't use observations in a regression where one of the regressors has some missing values (this will create, in effect, different-length vectors, causing you to have a non-conformable matrix problem). We can also do this in a little bit of a less brute-force way. Rather than removing *all* observations with missing values anywhere in a dataset, you might want to subset your data into a new dataset where there are no missing observations *in one variable*. We can do this too, using the `is.na()` function in conjunction with the `filter()` function:

```
someNAPSdata <- filter(psData, is.na(GDP) == FALSE)
someNAPSdata

## Source: local data frame [189 x 6]
##
##                country country_code        CO2          GDP
##                  (chr)        (chr)      (dbl)        (dbl)
## 1          Afghanistan          AFG   8470.770  10243250247
## 2              Albania          ALB   4415.068  10498760062
## 3              Algeria          DZA 119276.509 116511000000
## 4              Andorra          ADO    517.047   2829050839
## 5  Antigua and Barbuda          ATG    524.381   1012089673
## 6            Argentina          ARG 179000.938 291705000000
## 7              Armenia          ARM   4217.050   5918256387
## 8            Australia          AUS 368170.467 797778000000
## 9              Austria          AUT  67975.179 335532000000
## 10          Azerbaijan          AZE  30678.122  28310397534
## ..                 ...          ...        ...          ...
## Variables not shown: Pop (int), ones (dbl)
```

This grabs only the rows where GDP isn't missing. There are many other data-cleaning type operations in `R`, and we'll explore more of them as the semester continues.

Finally, if you're using `knitr`, I find that writing `<<message = FALSE, warning = FALSE>>=` at the beginning of your chunks can make your output nicer.

I hope that was helpful. Let's get back to it.

Off we go

This week, we'll go over how to do hypothesis testing in `R` - aka, how to tell if the point estimate you've estimated is statistically significantly different from...something. We usually use zero. This is obviously super important. If time permits, we'll also think about running regressions with dummy variables and interaction terms. To start off with, we will of course load our usual packages, our favorite dataset, and a function we wrote last week. I find it helpful to give my `R` files a "preamble" that looks something like this:

```
####### PACKAGES #######
library(dplyr)
library(lfe)
library(readr)


####### FUNCTIONS #######
tbldfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

OLS <- function(data, y, X) {
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  return(betahat)
}
```

Notice that we've renamed our `noIntOLS()` function from last week `OLS()` to make my typing life easier.

We're also going to install and then load one new package that will be helpful later:

```
install.packages('magrittr')
```

```
library(magrittr)
```

Just for fun, we'll practice grabbing our `autos` dataset from the .csv instead of from the .dta. Note that when we do this, we'll use `read_csv()`, a function from the `readr` library. It works just like the `read_dta()` function from `haven`, importing data directly as `tbl_df` objects. This should be old hat at this point.[5]

```
autos <- read_csv("autos.csv")
```

This particular CSV already had column names - but if they'd been left off for some reason, we could add them back in with the following command:

[5] This is a weird saying. Old hats are gross and sweaty. Our mad `R` skillz are neither gross nor sweaty.

```
names(autos) <- c("make", "price", "mpg",
                  "rep78", "headroom", "trunk",
                  "weight", "length", "turn",
                  "displacement", "gear_ratio", "foreign")
```

Let's start by adding a column of ones to our `autos` dataset:

```
autos <- mutate(autos, ones = 1)
autos
```

```
## Source: local data frame [74 x 13]
##
##             make price   mpg rep78 headroom trunk weight length
##            (chr) (int) (int) (int)    (dbl) (int)  (int)  (int)
## 1    AMC Concord  4099    22     3      2.5    11   2930    186
## 2      AMC Pacer  4749    17     3      3.0    11   3350    173
## 3     AMC Spirit  3799    22    NA      3.0    12   2640    168
## 4  Buick Century  4816    20     3      4.5    16   3250    196
## 5  Buick Electra  7827    15     4      4.0    20   4080    222
## 6  Buick LeSabre  5788    18     3      4.0    21   3670    218
## 7     Buick Opel  4453    26    NA      3.0    10   2230    170
## 8    Buick Regal  5189    20     3      2.0    16   3280    200
## 9  Buick Riviera 10372    16     3      3.5    17   3880    207
## 10 Buick Skylark  4082    19     3      3.5    13   3400    200
## ..           ...   ...   ...   ...      ...   ...    ...    ...
## Variables not shown: turn (int), displacement (int), gear_ratio
##   (dbl), foreign (int), ones (dbl)
```

Ok, good.

[ARE 212] Students' *t*

Fun fact: Student's t distribution was discovered by a brewer at Guinness.[6] As if you needed another reason to think brewers are awesome.[7] Aaaanyway.

Suppose we're interested in testing the null hypothesis that β_j, the jth element of our $\boldsymbol{\beta}$ vector, is equal to $\bar{\gamma}$ against the alternative hypothesis that β_j is not equal to $\bar{\gamma}$, using a test with significance level α.[8]
As per Max's notes, the formula for the $t-test$ test statistic is:

$$t_j \equiv \frac{(b_j - \bar{\gamma})}{\sqrt{s^2 \cdot (X'X)^{-1}_{jj}}} = \frac{(b_j - \bar{\gamma})}{se(b_j)}$$

where b_j is our estimate of β_j, s^2 is our estimate of σ^2, and $(X'X)^{-1}_{jj}$ is the jth diagonal element of $(\mathbf{X}'\mathbf{X})^{-1}$. Let's write a function that will calculate this test statistic for us:

[6] Check it.
[7] Did I mention that I made the beer for my wedding?
[8] Holy run-on sentence, Batman!

```
tStat <- function(data, y, X, gamma) {
  # n and k for calculating degrees of freedom
  n <- nrow(data)
  k <- length(X)

  # turn the data into matrix objects
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)

  # run OLS
  betahat <- OLS(data, y, X)

  # calculate the residuals
  e <- ydata - xdata %*% betahat

  # calculate s^2
  s2 <- (t(e) %*% e) / (n-k)

  # (X'X)^-1
  XpXinv <- solve(t(xdata) %*% xdata)

  # standard error!
  se <- sqrt(s2 * diag(XpXinv))

  # the t statistic
  tStat <- (betahat - gamma) / se

  return(tStat)
}
```

Check out all of the intermediate steps here. I put these in because I could've written:

```
se <- sqrt((t(ydata - xdata %*% betahat) %*% (ydata - xdata %*% betahat) / (n - k))
           * diag(solve(t(xdata) %*% xdata)))
```

Gross, gross, gross. Do *not* do this. It will make your GSI and your professor very annoyed with you. If that isn't enough incentive, it will *also* make your code impossible to read and debug, not to mention hard to comment to remind yourself what it's doing. Friends don't let friends write code without intermediate steps.[9]

Let's take our `tStat()` function out for a test drive, on...you guessed it - a regression of car price on an intercept, mpg, and weight[10]. We'll do the normal economist thing and compare our $\hat{\beta}$ coefficients against zero:

[9] You could also have made this slightly nicer using the `dplyr` chaining commands. Fun fact - you can chain multiplication by writing something like:

```
a <- b * c %>%
    "*"(d)
```

This will get you `a = b * c * d`. Still, though, for things like what's inside the `tStat()` function, I think it's much more legible to do what we've done here - but chaining non-functions like that can sometimes be useful. If this is over your head, ignore it.

[10] I promise we'll use a more interesting dataset soon. Really. Truly. Madly. Deeply. #SavageGarden. None of that "One Direction" junk.

```
mytStat <- tStat(autos, "price", c("ones", "mpg", "weight"), 0)
```

For comparison, let's grab the t-statistic from the same regression, calculated using `felm()`. Note: I'm not sure why `knitr` is evaluating my commented-out $ signs as £ signs instead. Sometimes `R` is weird. But those should be $ signs.

```
# run the regression
cannedtStat <- felm(data = autos, price ~ mpg + weight) %>%
# save the summary
  # remember the magrittr package? That lets us do "%£%" to chain
  #  dollar sign operations
  # normally we'd call summary£coefficients[,3]
  summary() %$%
  (coefficients)[,3]
```

Now look at these two:

```
mytStat

##             price
## ones    0.5410180
## mpg    -0.5746808
## weight  2.7232382

cannedtStat

## (Intercept)         mpg      weight
##   0.5410180  -0.5746808   2.7232382
```

Excellent. But outputting just a t-statistic is kind of silly. We'd actually like our function to return all of the same things as `felm()` does: $\hat{\beta}$, the standard error, the t-statistic, and the p-value (denoted in the `felm()` summary as `Pr(>|t|)`). So let's write that up. We're going to make one simplification - we'll eliminate $\bar{\gamma}$. 99.999% of the time in economics, we test a null hypothesis that a coefficient is equal to zero - so we'll implmement that directly here.[11]. To add the p-value, note that it's defined as:

$$p = Pr(t > |t_j|) * 2$$

```
olsPlus <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  betahat <- OLS(data, y, X)
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
```

[11] Feel free to adapt this and put $\bar{\gamma}$ back into your own version. I won't tell.

```
  se <- sqrt(s2 * diag(XpXinv))
  tStat <- (betahat - 0) / se

  # the p value
  # grab the t stat
  pVal <-  tStat %>%
    # take its absolute value
     abs() %>%
    # multiply by negative 1
    "*"(-1) %>%
    # compare it to a t distr with DOF n-k
    pt(df = n - k) %>%
    # and multiply that by 2
    "*"(2)

  olsOut <- list(betahat, se, tStat, pVal)
  names(olsOut) <- c("betahat", "SE", "tStat", "pVal")
  return(olsOut)
}
```

And again, test:

```
myFancyOLS <- olsPlus(autos, "price", c("ones", "mpg", "weight"))
myCannedOLS <- felm(data = autos, price ~ mpg + weight) %>%
  summary() %>%
  "$"(coefficients)

myFancyOLS

## $betahat
##               price
## ones   1946.068668
## mpg     -49.512221
## weight    1.746559
##
## $SE
## [1] 3597.0495988   86.1560389    0.6413538
##
## $tStat
##             price
## ones    0.5410180
## mpg    -0.5746808
## weight  2.7232382
##
## $pVal
##               price
## ones   0.590188628
## mpg    0.567323727
```

```
## weight 0.008129813

myCannedOLS

##                Estimate   Std. Error    t value    Pr(>|t|)
## (Intercept) 1946.068668 3597.0495988  0.5410180 0.590188628
## mpg          -49.512221   86.1560389 -0.5746808 0.567323727
## weight         1.746559    0.6413538  2.7232382 0.008129813
```

Boooooooooom. Anything the canned routine can do, we can do also![12]

All together now: F tests

Calculating a test statistic and p-value for each coefficient individually is nice, but we can actually go one step further and do a test of *joint* significance by calculating the F statistic.[13] This statistic is a measure of whether a bunch of different coefficients are jointly different from their hypothesized value under the null (usually zero). Suppose we have the following data generating process:

$$\mathbf{y} = \alpha + \mathbf{x_1}\beta_1 + \mathbf{x_2}\beta_2 + \mathbf{x_3}\beta_3 + \varepsilon$$

To test whether β_1, β_2, and β_3 are jointly different from a zero vector, we need to set up two objects: a matrix indicating which coefficients we're testing, and a zero vector of appropriate size. We'll call these objects $\mathbf{R}$ and $\mathbf{r}$, respectively:

$$\mathbf{R} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{r} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Notice what we're doing here. The $\mathbf{R}$ matrix says which coefficient we're restricting (notice that we're never restricting the intercept - what would that mean?) on each line, and the $\mathbf{r}$ vector describes the restrictions we're actually making. If we wanted to test whether coefficients were jointly different from, say, 0, 2, and 5, we'd set:

$$\mathbf{r} = \begin{bmatrix} 0 \\ 2 \\ 5 \end{bmatrix}$$

We can write out the F test as:

$$\mathbf{F} \equiv \frac{(\mathbf{Rb} - \mathbf{r})'[\mathbf{R}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{R}']^{-1}(\mathbf{Rb} - \mathbf{r})/J}{s^2} = \frac{(\mathbf{Rb})'[\mathbf{R}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{R}']^{-1}(\mathbf{Rb})/J}{s^2}$$

where the equality comes from setting $\mathbf{r} = \mathbf{0}$, and where $J = 3$ because we're making 3 restrictions.
Now let's implement this test in an expanded version of our OLS function. We could do this one of two ways: we could actually include as inputs into our function $\mathbf{R}$ and $\mathbf{r}$, or we could make the (usual) assumption that we'll be testing that each coefficient is equal to zero. We'll do the latter. We'll also calculate the p-value associated with the F statistic. Note: the way we're going to set this up necessitates that our `X` vector always includes an intercept, and that the intercept is the first term in the `X` vector. This should give you slight heebie-jeebies, but we'll live with it for now:

[12] Sort of. Not really. And definitely not better. But let's soak in the moment of victory for now.

[13] Check out Max's notes for a great explanation of why this is *not* the same thing as running separate t tests on each coefficient in your model.

```
megaOLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  betahat <- OLS(data, y, X)
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  se <- sqrt(s2 * diag(XpXinv))
  tStat <- (betahat - 0) / se
  pVal <-  tStat %>%
     abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*"(2)

  # do the F test

  # nr of restrictions = nr of X variables (- intercept)
  J <- length(X) - 1
   # create an identity matrix of size k X k
  bigR <- diag(J) %>%
    # and attach a column of zeros at the front of it
    cbind(0, .)

  # intermediatestep
  RXXinvR <- solve(bigR %*% solve(t(xdata) %*% xdata) %*% t(bigR))

  # f statistic
  FStat <- t(bigR %*% betahat) %*% RXXinvR %*% (bigR %*% betahat) / (s2 * J)
  # p value for the F stat
  pValF <- 1 - pf(FStat, df1 = J, df2 = n-k)

  olsOut <- list(betahat, se, tStat, pVal, FStat, pValF)
  names(olsOut) <- c("betahat", "SE", "tStat", "tStatPVal", "FStat", "FStatPVal")
  return(olsOut)
}
```

As an aside (we'll skip over this in section), if you wanted to do this in a way that actually allowed you to input your own **R** and **r** objects, you could write:

```
megaOLSCustom <- function(data, y, X, bigR, littler) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
```

```
  betahat <- OLS(data, y, X)
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  se <- sqrt(s2 * diag(XpXinv))
  tStat <- (betahat - 0) / se
  pVal <-  tStat %>%
     abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*"(2)

  # do the F test

  # nr of restrictions = nr of X variables (- intercept)
  J <- length(littler)

  # intermediatestep
  RXXinvR <- solve(bigR %*% solve(t(xdata) %*% xdata) %*% t(bigR))


  # f statistic
  FStat <- t(bigR %*% betahat) %*% RXXinvR %*% (bigR %*% betahat) / (s2 * J)
  # p value for the F stat
  pValF <- 1 - pf(FStat, df1 = J, df2 = n-k)

  olsOut <- list(betahat, se, tStat, pVal, FStat, pValF)
  names(olsOut) <- c("betahat", "SE", "tStat", "tStatPVal", "FStat", "FStatPVal")
  return(olsOut)
}
```

Anyway - back to out slightly more restrictive version - let's test this against the canned routine. This time, we'll ask for only the F-stat to be returned:

```
myFStat <- megaOLS(autos, "price", c("ones", "mpg", "weight")) %>%
  "$"(FStat)

myFpVal <- megaOLS(autos, "price", c("ones", "mpg", "weight")) %>%
  "$"(FStatPVal)

myCannedFStat <- felm(data = autos, price ~ mpg + weight) %>%
  summary() %>%
  "$"(fstat)

myCannedFpVal <- (felm(data = autos, price ~ mpg + weight) %>%
  summary() %>%
  "$"(F.fstat))[4]
```

```
myFStat

##          price
## price 14.73982

myFpVal

##              price
## price 4.424878e-06

myCannedFStat

## [1] 14.73982

myCannedFpVal

##            p
## 4.424878e-06
```

Bingo. Our OLS function is all grown up! But there's still this annoying problem of the intercept. What if we don't input an intercept? We'd like OLS to warn us that our results might be weird. We can implement this quite easily in `R` with an `if` statement:

```
smarterOLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  # warn the user
  olsWarn <- "Intercept found"
  if (identical(xdata[,1], rep(1, n)) == FALSE)  {
    olsWarn <- "NO INTERCEPT DETECTED"
    print(olsWarn)
  }
  # beta
  betahat <- OLS(data, y, X)
  # resids
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  #std error
  se <- sqrt(s2 * diag(XpXinv))
  # t statistic & its pvalue
  tStat <- (betahat - 0) / se
  pVal <-  tStat %>%
     abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
```

```
    "*"(2)
  #Calculate F stat & its pvalue
    J <- length(X) - 1
    bigR <- diag(J) %>%
      cbind(0, .)
    RXXinvR <- solve(bigR %*% solve(t(xdata) %*% xdata) %*% t(bigR))
  FStat <- t(bigR %*% betahat) %*% RXXinvR %*% (bigR %*% betahat) / (s2 * J)
  pValF <- 1 - pf(FStat, df1 = J, df2 = n-k)

  olsOut <- list(betahat, se, tStat, pVal, FStat, pValF, olsWarn)
  names(olsOut) <- c("betahat", "SE", "tStat", "tStatPVal", "FStat", "FStatPVal", "intWarn")
  return(olsOut)

}
```

Let's give it a try:

```
(goodOLS <- smarterOLS(autos, "price", c("ones", "mpg", "weight")))
```

```
## $betahat
##              price
## ones   1946.068668
## mpg     -49.512221
## weight    1.746559
##
## $SE
## [1] 3597.0495988   86.1560389    0.6413538
##
## $tStat
##             price
## ones    0.5410180
## mpg    -0.5746808
## weight  2.7232382
##
## $tStatPVal
##              price
## ones   0.590188628
## mpg    0.567323727
## weight 0.008129813
##
## $FStat
##          price
## price 14.73982
##
## $FStatPVal
##              price
## price 4.424878e-06
##
```

```
## $intWarn
## [1] "Intercept found"

(badOLS <- smarterOLS(autos, "price", c("mpg", "weight")))

## [1] "NO INTERCEPT DETECTED"
## $betahat
##            price
## mpg    -5.479372
## weight  2.076235
##
## $SE
## [1] 28.1223701  0.1990485
##
## $tStat
##             price
## mpg    -0.1948403
## weight 10.4308004
##
## $tStatPVal
##               price
## mpg    8.460667e-01
## weight 4.818756e-16
##
## $FStat
##          price
## price 108.8016
##
## $FStatPVal
##              price
## price 4.440892e-16
##
## $intWarn
## [1] "NO INTERCEPT DETECTED"
```

Okay, that makes me feel a little bit better. We could've actually gone one step further than we did. Right now, we've just added a warning to our function - but we could instead make our function break if it doesn't find an intercept:

```
smartestOLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbldfGrabber(data, y)
  xdata <- tbldfGrabber(data, X)
  # warn the user
  if (identical(xdata[,1], rep(1, n)) == FALSE)  {
    olsWarn <- "NO INTERCEPT DETECTED"
    stop(olsWarn)
```

```
  }
  # beta
  betahat <- OLS(data, y, X)
  # resids
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  #std error
  se <- sqrt(s2 * diag(XpXinv))
  # t statistic & its pvalue
  tStat <- (betahat - 0) / se
  pVal <-  tStat %>%
     abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*"(2)
  #Calculate F stat & its pvalue
    J <- length(X) - 1
    bigR <- diag(J) %>%
      cbind(0, .)
    RXXinvR <- solve(bigR %*% solve(t(xdata) %*% xdata) %*% t(bigR))
  FStat <- t(bigR %*% betahat) %*% RXXinvR %*% (bigR %*% betahat) / (s2 * J)
  pValF <- 1 - pf(FStat, df1 = J, df2 = n-k)

  olsOut <- list(betahat, se, tStat, pVal, FStat, pValF)
  names(olsOut) <- c("betahat", "SE", "tStat", "tStatPVal", "FStat", "FStatPVal")
  return(olsOut)

}
```

Notice what we've done here: we added a `stop()` command - this breaks the function, and outputs the text in quotes. Check it:

```
(goodOLS <- smartestOLS(autos, "price", c("ones", "mpg", "weight")))

## $betahat
##               price
## ones   1946.068668
## mpg     -49.512221
## weight    1.746559
##
## $SE
## [1] 3597.0495988   86.1560389    0.6413538
##
## $tStat
##              price
## ones    0.5410180
```

```
## mpg    -0.5746808
## weight  2.7232382
##
## $tStatPVal
##              price
## ones   0.590188628
## mpg    0.567323727
## weight 0.008129813
##
## $FStat
##          price
## price 14.73982
##
## $FStatPVal
##              price
## price 4.424878e-06
```

```
(badOLS <- smartestOLS(autos, "price", c("mpg", "weight")))
```

```
## Error in smartestOLS(autos, "price", c("mpg", "weight")):  NO INTERCEPT DETECTED
```

See? It won't even let us run that final chunk of code, and it also tells us why. Niiiiice.

Bonus: Dummies for dummies

So far, we've basically been running the same boring regression. Let's digress to regress something else.[14] Let's think about dummy variables, which is just a fancy (?) name for variables that only ever take on the values 1 or 0.

Suppose we're interested in figuring out the auto price effect (okay, we're not digressing *that* far) of being foreign, having above-median MPG, and being both foreign and high-MPG.

To do this, we'll first add an indicator variable to our `autos` dataset, equal to one if MPG is above median, and zero otherwise. To do that, we'll first find the median MPG in the data.

```
#The summarize() command lets us grab the median of the dataset
medianMPG <- summarize(autos, median(mpg)) %>%
  # but it returns it as tbl_df; we want a scalar, so let's make one
  as.numeric()
```

The median is 20. Cool. We can use that fact, in conjunction with the `ifelse()` function, to create a new variable (we'll save over our `autos` dataset):

```
# ifelse takes in a statement to evaluate,
  # a value to return if true, and a value to return if false
```

[14] See what I did there?

```
#this line adds the highMpg variable to our dataset, = 1 if MPG >= median and 0 otherwise
autos <- mutate(autos, highMpg = ifelse(mpg >= medianMPG, 1, 0))
```

We can proceed in two ways: an ugly way, and an elegant way. The ugly way is easier to interpret, so we'll start there. The ugly way is the following: split our dataset into domestic and foreign cars, and run

$$p_i = \alpha + \beta_1 \mathbb{1}[\text{High MPG}]_i + \nu$$

(where p_i is a car's price) on both parts. We can then compare the results from the regression on the domestic group with the same regression on the foreign group to get at the additional benefit (or cost) from increased fuel efficiency from foreign vs. domestic cars. Let's do it:

```
regOuts <- sapply(0:1, function(x) {
  # filter grabs rows of autos where highMpg == x
  filter(autos, foreign == x) %>%
    # and then we run the regression
    smartestOLS("price", c("ones", "highMpg"))
})

regOuts[1,]

## [[1]]
##              price
## ones     6800.967
## highMpg -1722.012
##
## [[2]]
##              price
## ones     9258.600
## highMpg -3719.188
```

Whoa! The average price for domestic cars is lower than for foreign cars! 6,801 vs. 9,259, which we can see from the intercept. But notice something else interesting: making your car fuel efficient lowers prices for foreign manufacturers much more than it does for domestic manufacturers (-3,719 vs. -1,722). The differential effect of high MPG for foreign relative to domestic cars is 1,997, which seems like a lot (although again, units????).

So that's the brute-force way. But there are two costs to specifying this model this way: first, we're running two regressions, and then doing calculations, where we could be running one regression. Second, we don't have standard errors, so we can't actually tell if the high-MPG effect is statistically different between domestic and foreign manufacturers. Hence the elegant way: put everything into one regression.

We do this with an *interaction term*, which is exactly what it sounds like: $\mathbb{1}[\text{Foreign}] \times \mathbb{1}[\text{High MPG}]$. So let's make that in our data:

```
autos <- mutate(autos, interact = foreign * highMpg)
```

Now we can run the following regression:

$$p_i = \alpha + \beta_1 \mathbb{1}[\text{Foreign}] + \beta_2 \mathbb{1}[\text{High MPG}] + \beta_3 \mathbb{1}[\text{Foreign}] \times \mathbb{1}[\text{High MPG}] + \varepsilon_i$$

```
myInteractions <- smartestOLS(autos, "price", c("ones", "foreign", "highMpg", "interact"))
```

Let's compare the coefficients from this regression to what we got from running seperate regressions earlier:

```
regOuts[1,]

## [[1]]
##             price
## ones     6800.967
## highMpg -1722.012
##
## [[2]]
##             price
## ones     9258.600
## highMpg -3719.188

myInteractions$betahat

##              price
## ones      6800.967
## foreign   2457.633
## highMpg  -1722.012
## interact -1997.176
```

This is cool: we can learn a couple of things. First, notice that the coefficient on the intercept and on `highMpg` are the same as they were in the regression on only the domestic data. Next, note that the coefficient on `foreign`, which we couldn't have in our regression earlier for obvious reasons[15], is exactly the same as the intercept from the regression on the foreign data minus the intercept from the regression on the domestic data. In other words, we can figure out the average price for foreign cars in two ways: by calculating the intercept from a regression of foreign cars only, *or* from calculating $\alpha + \beta_1$ from the combined regression. The average price for domestic cars is just α.

We can finally interpret the coefficient on `interact`: this is our -1997 value from before: the additional price hit from high fuel efficiency, for foreign cars relative to their domestic counterparts. Cool!

Now that we know how to interpret this bad boy, let's also check to see if it's statistically significant:

```
myInteractions$tStat

##              price
## ones     13.381021
## foreign   1.827623
## highMpg  -2.203761
## interact -1.234712
```

[15] If these reasons aren't obvious to you, think about it for a second. Got it? Good.

That's a t-statistic of -1.2. Not statistically significant. So actually, we can't discern a differential effect of "switching" from fuel inefficient to fuel efficient for foreign vs. domestic automakers.[16]
All this to say: there are, as usual, multiple ways of getting to the same answer in `R` and in econometrics in general. I also often forget how to interpret interaction terms, so running the two-step procedure is a helpful reference point for me.[17]

We'll leave it here. Next week, we'll pick up with `ggplot2`.

[16] Why do I put "switching" in quotes? Nothing here is randomly assigned, so all we're actually measuring is a correlation, rather than a causal relationship. Don't worry - Michael Anderson will show you the causal inference light next semester.

[17] When we start introducing fixed effects into the mix, this becomes a little more convoluted, since running separate regressions will often involve including different sets of fixed effects. That probably won't make sense for now - but don't say I didn't warn you.